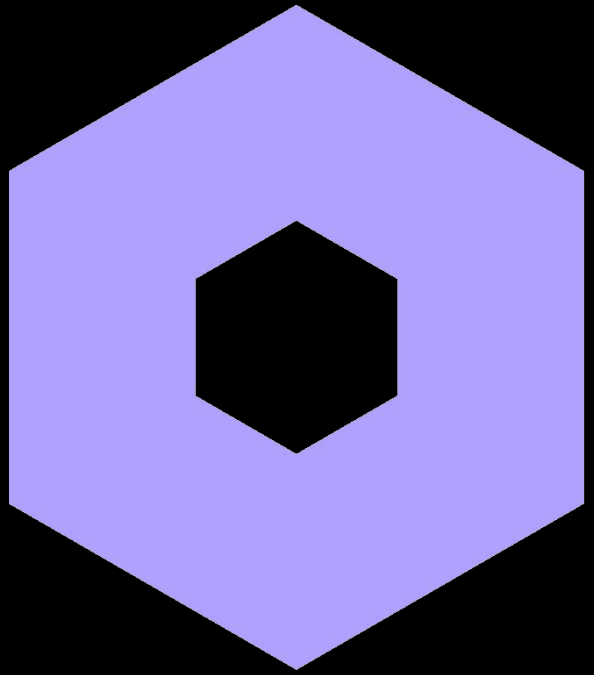
JANUARY 24



Diabetes data

Project Description

- Regression
- Classification
- Clustering

Goal of this project

- Analyze complex data sets
- Clean it
- Apply machine learning model
- Make predictions

Problem overview

- Many features
- Categorial features
- Irrelevant columns
- Imblanced data

```python
## Loading the data

import pandas as pd
# Reload the dataset as a Pandas DataFrame
file_path = r"C:\Users\Asus\Desktop\diabetic_data.csv"
data = pd.read_csv(file_path)
categorial=data.select_dtypes(include=['object','category']).columns
data = data.drop(['max_glu_serum','weight', 'A1Cresult','payer_code','patient_nbr','encounter_id','metformin-pioglitazone','metformin-rosiglitazone','glimepiride-pioglitazone'
|   |   |   |   |   ,'glipizide-metformin','glyburide-metformin','citoglipton','examide','tolazamide','troglitazone','miglitol','acarbose','rosiglitazone','pioglitazone','tolbutamide','glyburide','acetohexamide',
|   |   |   |   |   'glimepiride','chlorpropamide','nateglinide','repaglinide','diag_3','diag_2','diag_1','metformin','glipizide','medical_specialty',],axis=1)
data = data.dropna()
data
```

Python

```python
data['diabetesMed']=data['diabetesMed'].map({'No':0,'Yes':1})
data['change']=data['change'].map({'No':0,'Ch':1})
data['insulin']=data['insulin'].map({'No':0,'Steady':1,'Down':2,'Up':3})
data['readmitted']=data['readmitted'].map({'NO':0,'>30':1,'<30':1})
```

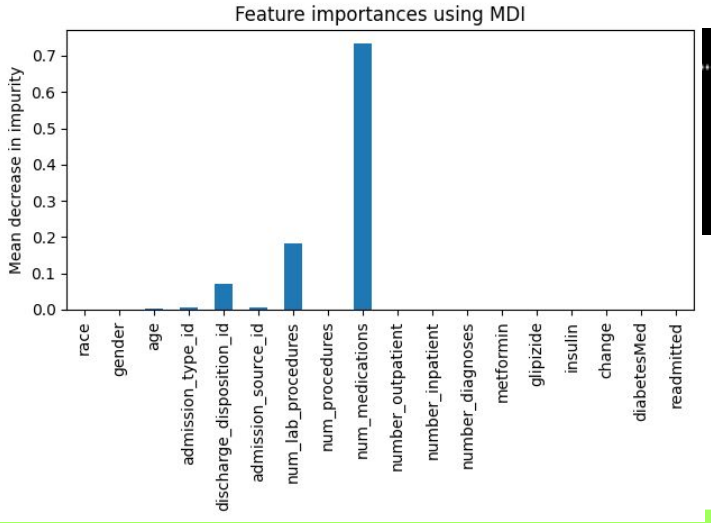
```python
from sklearn.preprocessing import StandardScaler
# Standardising column names and remove duplicates
scaler=StandardScaler()
scaled_data = scaler.fit_transform(data)

scaled_df=pd.DataFrame(scaled_data,columns=data.columns)

# Applying standard scaling to numerical features for clustering
scaled_df
```

```python
# Defining features (X) and target (y)
x = data[['num_medications','num_lab_procedures','discharge_disposition_id']]
y = data['time_in_hospital'] #it should be numerical

#handling missing value
```



	Model	MAE	MSE
0	Linear Regression	1.906336	6.261203
1	Decision Tree	1.914441	6.350824
2	Gradient Boosting	1.874660	6.137249
3	XGBoost	1.881129	6.184145

Model overview

Regression :

- Linear regression
- Decision tree
- Gradient Boosting Regressor
- XGBoost Regressot

Regression is a **supervised learning technique** used to predict continuous numerical outcomes based on input features. It identifies the relationship between independent variables and a dependent variable. Examples include linear regression, decision tree regression, and XGBoost regression.

```python
# importing requires sklearn packages
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Defining target and features
x = data.drop('readmitted',axis=1)
y=data['readmitted']#target



# Spliting data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.3,random_state=42)
```

```python
# importing requires sklearn packages
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score

# Initialising and train Random Forest Classifier
randomforest=RandomForestClassifier(criterion='gini',max_depth=5, min_samples_leaf=1,min_samples_split=2,n_estimators=100,random_state=42,class_weight='balanced')

randomforest.fit(x_train,y_train)

# Predictions
y_pred=randomforest.predict(x_test)

# Evaluation
accuracy_r =accuracy_score(y_test,y_pred)
precision_r=precision_score(y_test,y_pred)
recall_r=recall_score(y_test,y_pred)
f1_r = f1_score(y_test,y_pred)
print(f"Random Forest Accuracy: {accuracy_r}")
print(f"Random Forest precision: {precision_r}")
print(f"Random Forest recall: {recall_r}")
print(f"Random Forest f1: {f1_r}")
```

```
[119]
...  Random Forest Accuracy: 0.6274933673970718
     Random Forest precision: 0.617056856187291
     Random Forest recall: 0.5102105934907466
     Random Forest f1: 0.558570097810899
```

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.61	0.59	0.53	0.55
Random Forest	0.62	0.61	0.51	0.55
K-Nearest Neighbors	0.58	0.55	0.52	0.53

Classification

- Logistic Regression
- Random Forest
- K-nearest Neighbors

Classification is a **supervised learning technique** used to predict categorical outcomes, such as binary or multi-class labels. It assigns input data into predefined categories. Examples include Logistic Regression, Random Forest Classifier, and XGBoost Classifier.

Clustering

```python
from sklearn.preprocessing import StandardScaler
# Standardising column names and remove duplicates
scaler=StandardScaler()
scaled_data = scaler.fit_transform(data)

scaled_df=pd.DataFrame(scaled_data,columns=data.columns)

# Applying standard scaling to numerical features for clustering
scaled_df
```

Clustering Method	Silhouette Score
K-Means	0.113
Hierarchical	0.334
DBSCAN	0.311

Clustering is an **unsupervised learning technique** used to group data points based on their similarity. It doesn't use labeled data and helps uncover hidden patterns or relationships in the dataset. Examples include K-means, DBSCAN, and hierarchical clustering.

3.3.4: Implement Random Hierarchical Clustering

```python
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
import numpy as np

df_sampled = resample(scaled_df, n_samples=20000, random_state=42)
data_pca=pca.fit_transform(df_sampled)
X2d = np.array(data_pca)
# Initialising and fiting hierarchical clustering
hierarchical = AgglomerativeClustering(n_clusters=2)
hierarchical_labels= hierarchical.fit_predict(X2d)

# Evaluating with Silhouette Score
hierarchical_silhouette =silhouette_score(X2d,hierarchical_labels)
print(f"Hierarchical Clustering Silhouette Score: {hierarchical_silhouette}")
```

```
Hierarchical Clustering Silhouette Score: 0.33484215228322717
```

3.3.5: Implement DBSCAN for Density-Based Clustering

```python
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
import numpy as np

# Initialising and fiting DBSCAN
dbscan = DBSCAN(eps=0.5,min_samples=10)
dbscan_labels = dbscan.fit_predict(scaled_df)

# Filtering noise points (labeled as -1)
core_samples_mask = dbscan_labels != -1
scaled_df_core = scaled_df[core_samples_mask]
labels_core = dbscan_labels[core_samples_mask]

# Evaluate with Silhouette Score (excluding noise)
if len(set(labels_core)) > 1:  # Silhouette Score is only valid with at least 2 clusters
    dbscan_silhouette = silhouette_score(scaled_df_core, labels_core)
    print(f"DBSCAN Clustering Silhouette Score (excluding noise): {dbscan_silhouette:.2f}")
else:
    print("Silhouette Score cannot be computed with less than 2 clusters (excluding noise).")
```

```
DBSCAN Clustering Silhouette Score (excluding noise): 0.31
```

	Model	MAE	MSE
0	Linear Regression	1.906336	6.261203
1	Decision Tree	1.914441	6.350824
2	Gradient Boosting	1.874660	6.137249
3	XGBoost	1.881129	6.184145

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.61	0.59	0.53	0.55
Random Forest	0.62	0.61	0.51	0.55
K-Nearest Neighbors	0.58	0.55	0.52	0.53

Clustering Method	Silhouette Score
K-Means	0.113
Hierarchical	0.334
DBSCAN	0.311

Gradient Boosting is an advanced model that builds multiple small models one after another, each correcting the mistakes of the previous one. It's powerful for making accurate predictions.

Random Forest uses many decision trees to classify data. It combines their results to make predictions, making it reliable, accurate, and resistant to overfitting.

Hierarchical Clustering groups data points into a tree-like structure based on their similarity. It's great for finding patterns and doesn't require pre-setting the number of clusters.

Final Analysis and Conclusion

To achieve these scores, I reduced my features from 50 to 16 by removing columns with missing, NaN, and duplicate values. This step helped make the dataset cleaner and more efficient for modeling.

After tuning my models, the scores might improve further because tuning involves using grid search to find the best parameters for each model, ensuring optimal performance and results.

Overall, I was satisfied with the scores I achieved, except for clustering. If I had more time, I would investigate the issues in clustering to identify what went wrong and explore ways to achieve better performance in this area.